

Listas de conteúdos disponíveis em [ScienceDirect](https://www.sciencedirect.com)

Revisão de pesquisa educacional

página inicial da revista: www.elsevier.com/locate/edurev

Análise

Desmistificando o pensamento computacional

Valerie J. Calma^{a, *}, Chen Sun^a, Jodi Asbell-Clarke^b^a Universidade Estadual da Flórida, EUA^b TERC, EUA

informações do artigo

Historia do artigo:

Recebido em 4 de março de 2017

Recebido em formato revisado em 17 de julho de 2017

Aceito em 14 de setembro de 2017

Disponível on-line em 20 de setembro de 2017

Palavras-chave:

Pensamento computacional

Alfabetização computacional

Solução de problemas

Programação

abstrato

Este artigo examina o crescente campo do pensamento computacional (PC) na educação. A a revisão da literatura relevante mostra uma diversidade em definições, intervenções, avaliações e modelos. Depois de sintetizar várias abordagens usadas para desenvolver o construto em Nas configurações K-16, criamos a seguinte definição funcional de CT: O conceitual base necessária para resolver problemas de forma eficaz e eficiente (ou seja, algoritmicamente, com ou sem auxílio de computadores) com soluções reutilizáveis em diferentes contextos. Esta definição destaca que a PC é principalmente uma forma de pensar e agir, que pode ser exibida através do uso de habilidades específicas, que então podem se tornar a base para avaliações baseadas no desempenho de habilidades de PC. Com base na literatura, categorizamos a TC em seis facetas principais: decomposição, abstração, design de algoritmo, depuração, iteração, e generalização. Este artigo mostra exemplos de definições, intervenções, avaliações e modelos de PC em uma variedade de disciplinas, com um apelo para pesquisas mais extensas em esta área.

© 2017 Elsevier Ltd. Todos os direitos reservados.

Conteúdo

1. Introdução 1.1.	143
Definições de pensamento computacional.	143
1.2. Metas e foco	144
2. Método 2.1.	144
Procedimento	144
2.2. Critérios de inclusão e exclusão	144
3. Resultados da revisão da literatura	145
3.1. Características do pensamento computacional.	145
3.1.1. Componentes do CT	145
3.1.2. Diferenças entre PC e outros tipos de competências de pensamento	145
3.1.3. Relação da TC com a informática e a programação.	146
3.2. Intervenções para desenvolver o pensamento computacional.	147
3.2.1. Investigação em CT utilizando ferramentas de	147
programação 3.2.2. Investigação com	148
recurso à robótica 3.2.3. Investigação utilizando design de jogos e outras	148
ferramentas de intervenção 3.2.4. Habilidades de CT que fundamentam programação, robótica e design de jogos.	149

* Autor correspondente.

Endereço de e-mail: vshute@fsu.edu (VJ Shute).

3.3. Avaliação do pensamento computacional.	149
3.3.1. Avaliações baseadas em scratch	149
3.3.2. Avaliação baseada em jogos/simulações 3.3.3.	150
Balanças CT validadas para uso genérico.	150
3.4. Modelos de pensamento computacional 4.	150
Nossa definição e estrutura de pensamento computacional 4.1. Comparação	151
com outros modelos 4.2. Exemplos de modelos	152
emergentes de K-12 CT 5. Discussão	152
.....	154
Reconhecimentos	157
Referências	157

1. Introdução

Na Idade Média, apenas grupos selecionados de pessoas (por exemplo, sacerdotes e escribas) sabiam ler e escrever. Mas à medida que o mundo evoluiu, cada vez mais pessoas precisavam dessas habilidades. Hoje, o rápido aparecimento dos computadores no século XX está a forçar uma situação análoga: revolução onde a literacia digital é agora uma competência essencial para ter sucesso no nosso complexo mundo digital do século XXI. E embora nem todos precisamos nos tornar engenheiros de software, a maioria de nós usa computadores diariamente e precisa entender como comunicar-se com eles para aproveitar de forma mais eficaz seu poder de computação. Uma comunicação bem-sucedida nesse sentido é chamado pensamento computacional (TC).

Ao longo da última década, a TC tornou-se um tema muito quente na investigação e na prática educacional. Milhares de entradas aparecem em uma pesquisa geral no Google sobre sua definição, intervenções instrucionais e avaliação. Muitas dessas entradas sugerem que o TC está relacionado à codificação ou programação, mas considerar o TC como saber programar pode ser muito limitante. De acordo com Segundo o [Conselho Nacional de Pesquisa \(2010\)](#), todos deveriam adquirir TC, não apenas os programadores. As habilidades de CT incluem o gerenciamento de informações de forma eficaz e eficiente com tecnologias em nossa era orientada a dados ([Burke, O'Byrne, & Kafai, 2016](#); [Kim, Kwon, & Lee, 2014](#); [Lu e Fletcher, 2009](#); [Sanford e Naidu, 2016](#); [Asa, 2010](#)). Uma força de trabalho com indivíduos que possuam competências em TC aumenta a competitividade dos Estados Unidos no mercado econômico mundial ([NRC, 2010](#)).

Embora muitos programas afirmem ensinar competências de codificação, raramente analisam profundamente as formas de pensar utilizadas na TC. Análoga à investigação como forma de pensar cientificamente, a TC é um conjunto de práticas que estão entrelaçadas com formas de olhar para problemas que resultam em habilidades e entendimentos de PC. No entanto, não existe um currículo que construa uma base de compreensão da TC para os jovens alunos, como existe na matemática ou nas ciências. São necessários modelos que ajudem a destacar o TC dentro da corrente práticas de sala de aula, porque há muito poucas oportunidades para encaixar novos conteúdos nos currículos escolares existentes. Muitos problemas apresentados nos currículos atuais, no entanto, podem ser abordados com a TC.

Possuir boas habilidades de TC pode ser um motivador para os alunos seguirem a ciência da computação ([Allan, Barr, Brylow, & Hambrusch, 2010](#)) e outras especialidades relacionadas a STEM ([Sneider, Stephenson, Schafer, & Flick, 2014](#)). A CT também tem sido associada à criatividade e inovação ([Mishra, Yadav, & the Deep-Play Research Group, 2013](#); [Repenning et al., 2015](#)) e tem aplicações importantes em outras áreas STEM ([Barr & Stephenson, 2011](#); [Sengupta, Kinnebrew, Basu, Biswas, & Clark, 2013](#)). Uma definição exata de CT, no entanto, permanece indefinido ([Barr, Harrison, & Conery, 2011](#); [Grover & Pea, 2013](#)).

Neste artigo, discutimos as diversas definições de TC emergentes de diferentes disciplinas e apresentamos uma definição de TC em termos de como os educadores do ensino fundamental e médio podem pensar em construir uma base sólida para a TC em jovens alunos. Dividimos o CT em componentes mais frequentemente citados na literatura e propõem um modelo para incorporar a aprendizagem e avaliação de TC no ensino fundamental e médio currículos.

1.1. Definições de pensamento computacional

O pensamento computacional (PC) remonta ao trabalho construcionista de Seymour Papert ([Papert, 1980, 1991](#)) e foi inicialmente cunhado como termo em um artigo seminal de [Wing \(2006\)](#). Ela explicou que a CT envolve “resolver problemas, projetar sistemas e compreender o comportamento humano, recorrendo aos conceitos fundamentais da ciência da computação” ([Wing, 2006](#), p. 33). Como tal, representa uma capacidade de analisar e depois resolver vários problemas. Os seus argumentos forneceram uma nova perspectiva sobre a(s) relação(ões) entre humanos e computadores e deram origem a uma onda de investigação sobre TC.

A definição de TC mais citada vem de [Cuny, Snyder e Wing \(2010\)](#), observando que a TC é um processo de pensamento onde “... as soluções são representadas de uma forma que pode ser efetivamente executada por um agente de processamento de informações” (conforme citado em [Asa, 2010](#), pág. 1). Isto se refere não apenas a problemas bem estruturados, mas também a problemas mal estruturados (ou seja, problemas complicados da vida real). problemas cujas soluções não são definidas nem mensuráveis). Outros pesquisadores criaram suas próprias definições em relação às suas áreas específicas de pesquisa. Por exemplo, [Barr et al. \(2011\)](#) concluíram que no ensino fundamental e médio, o CT envolve resolução de problemas habilidades e disposições específicas, como confiança e persistência, ao enfrentar problemas específicos. [Berland e Wilensky \(2015\)](#) definiu a TC como “a capacidade de pensar com o computador como ferramenta” (p. 630) e sugeriu o uso de “perspectivas computacionais” como uma alternativa ao “pensamento computacional” para enfatizar que a TC pode ser restringida por contextos.

Além disso, a PC foi definida como "...estudantes que utilizam computadores para modelar as suas ideias e desenvolver programas" (Israel, Pearson, Tapia, Wherfel, & Reese, 2015, p. 264), ligando explicitamente a PC às competências de programação.

Conforme descrito acima, as definições de TC variam na sua operacionalização de TC em certos estudos e não são particularmente generalizáveis (por exemplo, Berland & Wilensky, 2015; Ioannidou, Bennett, Repenning, Koh, & Basawapatna, 2011; Israel et al., 2015).

A definição de TC está evoluindo à medida que os pesquisadores começam a agregar conhecimento sobre TC.

1.2. Metas e foco

Neste artigo, pretendemos explorar o âmbito e a complexidade da TC e estabelecer uma definição e um quadro claros que ajudarão no desenvolvimento da pedagogia e avaliação da TC, especialmente para o ensino fundamental e médio. No final, fornecemos exemplos de pesquisas atuais e ideias para pesquisas futuras. Nossa revisão aborda as seguintes questões: (1) Quais são as principais características e componentes da TC? (2) Que intervenções são (ou podem ser) utilizadas para treinar/aprimorar a PC? (3) Que tipos de medidas são utilizadas para avaliar o TC? e (4) Quais os principais referenciais/ modelos teóricos da PC? Mais uma vez, o nosso principal objectivo é derivar um modelo geral de CT que possa ser utilizado como quadro para avaliar e apoiar a CT. Nosso modelo é derivado de uma extensa revisão da literatura e serve para orientar o desenvolvimento da pedagogia e avaliação da TC em ambientes educacionais.

2. Método

2.1. Procedimento

Começamos por recolher artigos de investigação relevantes para a TC, incluindo publicações e anais revistos por pares, além de um relatório académico de um workshop nacional sobre TC. Pesquisamos várias bases de dados usando as palavras-chave "pensamento computacional" (aspas incluídas) e especificamos que o termo ocorria no título ou no resumo.

Os seguintes bancos de dados on-line e sites foram empregados neste esforço de coleta de pesquisa:

- o ERIC: O Centro de Informação de Recursos Educacionais (ERIC) consiste no Índice de Recursos em Educação e no Índice Atual de Revistas em Educação. ERIC é uma base de dados ampla e popular que contém relatórios educacionais, avaliações e pesquisas. o PsycINFO: Este site é hospedado pela American Psychological Association, que traz citações e resumos de estudos académicos artigos de periódicos, capítulos de livros, livros e dissertações, em psicologia e disciplinas afins.
- o JSTOR: Um banco de dados de edições anteriores de periódicos importantes nas áreas de humanidades, ciências sociais e ciências. A diferença entre o número publicado mais recentemente de qualquer periódico e a data do número mais recente disponível no JSTOR é de 2 a 5 anos. o Google Scholar: Este site foi utilizado para buscar e adquirir referências específicas. O Google Académico é um site que fornece artigos, teses, livros, resumos e artigos revisados por pares de editoras académicas, sociedades profissionais, repositórios de preprints, universidades e outras organizações académicas.

Iniciamos a linha do tempo em 2006, quando Wing publicou seu artigo seminal sobre CT, sinalizando o início de um corpus de pesquisas e projetos sobre o tema. Após a coleta de artigos relevantes, realizamos nova triagem dos artigos, classificando-os em artigos conceituais e estudos empíricos. Os documentos conceituais discutiram as características gerais da TC, fornecendo um enquadramento teórico ou sugerindo práticas de ensino para integrar a TC na educação. Os estudos empíricos tenderam a testar e justificar intervenções e medidas específicas de PC através de desenhos de investigação qualitativos e quantitativos. Por último, integramos os resultados da revisão da literatura com a investigação empírica para criar o nosso modelo de competências¹ para o PC, com o objetivo de facilitar e avaliar o PC em ambientes educativos.

2.2. Critérios de inclusão e exclusão

Ao todo, foram inicialmente coletados cerca de 70 documentos, tanto empíricos quanto teóricos. Desse conjunto, um total de mais de 45 documentos atenderam aos critérios para inclusão na revisão de literatura. Os critérios de inclusão consistiram na relevância dos documentos para os tópicos de pesquisa deste artigo (por exemplo, habilidades de pensamento computacional – características e processos, modelos, avaliações e intervenções). Foram incluídos estudos experimentais e não experimentais. Criamos uma tabela resumindo o conjunto completo de 70 artigos coletados e, em seguida, excluímos artigos da lista pelos seguintes motivos: (a) pesquisa de baixa qualidade descrita no artigo (por exemplo, declarações e suposições excessivas apresentadas com suporte inadequado), (b) tangencial ou nenhum foco especificamente em CT, (c) artigos empíricos que mediram algo diferente de CT como resultado, e (d) estudos piloto que eram de baixa qualidade e/ou relataram um pequeno tamanho de amostra.

¹ Um modelo de competências refere-se a um conjunto de conhecimentos, habilidades e outros atributos que compõem um construto específico (como PC). Responde à pergunta: O que você quer dizer sobre a pessoa no final da avaliação? As variáveis no modelo de competências são geralmente chamadas de "nós" e descrevem o conjunto de variáveis nas quais as inferências se baseiam.

3. Resultados da revisão da literatura

Nossa revisão consiste em quatro partes principais. Começamos abordando as características distintas da TC. A seguir, examinamos intervenções para apoiar a TC seguidas de avaliações de TC usadas no ensino fundamental e médio e no ensino superior. Terminamos com um resumo dos modelos de pesquisa em TC.

3.1. Características do pensamento computacional

Nesta seção, definimos PC e o distinguimos de outros tipos de pensamento (por exemplo, pensamento sistêmico e pensamento matemático). Também discutimos a relação da CT com a ciência da computação.

3.1.1. Componentes da TC

Wing (2006) argumentou que TC não significa pensar como um computador; mas sim envolver-se em cinco processos cognitivos com o objetivo de resolver problemas de forma eficiente e criativa. Esses incluem:

1. Reformulação do problema e Reformular um problema em um problema solucionável e familiar.
2. Recursão e Construir um sistema incrementalmente com base nas informações anteriores.
3. Decomposição do problema e Divida o problema em unidades gerenciáveis.
4. Abstração e Modelar os aspectos centrais de problemas ou sistemas complexos.
5. Testes sistemáticos e Tomar ações intencionais para obter soluções.

A abstração é o principal elemento subjacente à TC (Wing, 2008), onde as pessoas coletam informações relevantes (e descartam dados irrelevantes) de sistemas complexos para gerar padrões e encontrar pontos em comum entre diferentes representações (Wing, 2010). A abstração possui camadas, portanto é necessário definir cada camada e esclarecer as relações entre as camadas. Isto envolve: (a) abstração em cada camada, (b) abstração como um todo e (c) interconexão entre camadas. Por exemplo, definir um algoritmo é um tipo de abstração e a "abstração de um procedimento passo a passo para obter informações e produzir alguma saída desejada"

(Asa, 2008, p. 3718).

Além da abstração e reformulação de problemas, Barr et al. (2011) argumentaram que a CT também consiste em organização e análise de dados, automação, eficiência e generalização. Automação é fazer com que um processo ou sistema opere automaticamente; eficiência significa criar soluções ótimas; e a generalização envolve a aplicação de estratégias de TC para resolver novos problemas. Barr e colegas também incluíram certas disposições importantes para a CT, como confiança, persistência em relação à resolução de tarefas complexas e capacidade de trabalhar bem em equipe. Da mesma forma, a TC descrita por Bers, Flannery, Kazakoff e Sullivan (2014) inclui atividades de abstração, generalização e tentativa e erro. Eles enfatizam particularmente a importância da depuração (ou seja, identificar e corrigir erros quando as soluções não funcionam conforme o esperado).

Num relatório abrangente do Conselho Nacional de Pesquisa, a CT consiste em cinco elementos essenciais e universais em todos os domínios (NRC, 2010): (1) teste de hipóteses, (2) gestão de dados, (3) paralelismo, (4) abstração, e (5) depuração. Ao resolver um problema complexo em qualquer domínio, deve-se gerar e testar hipóteses sistematicamente para compreender como o sistema funciona. É impossível testar todas as possibilidades, por isso é importante selecionar os parâmetros corretos para testar. O gerenciamento de dados envolve a coleta de dados de várias fontes, o processamento de padrões de dados e a representação de dados de maneira significativa. Paralelismo refere-se ao processamento simultâneo de informações de múltiplas fontes ou dimensões. A abstração se concentra na modelagem do funcionamento de um problema/sistema complexo. Finalmente, a depuração refere-se a encontrar e corrigir erros após a construção de modelos específicos. Recentemente, Anderson (2016) explicou cinco componentes de CT: (1) decomposição do problema, (2) reconhecimento de padrões, (3) abstração (ou seja, generalização de padrões repetidos), (4) design de algoritmo para soluções e (5) avaliação de soluções (ou seja, depuração).

Com base na revisão anterior, os pesquisadores criaram habilidades constituintes de CT semelhantes. Os componentes comuns entre os pesquisadores são: decomposição, abstração, algoritmos e depuração. Em nosso modelo de competências, a TC consiste similarmente em decomposição, abstração, algoritmos, depuração, bem como iteração e generalização (ver definições detalhadas, subcategorias e justificativas na Seção 4).

3.1.2. Diferenças entre CT e outros tipos de habilidades de pensamento

Os investigadores também estão a estudar as diferenças e semelhanças entre a PC e outros tipos de pensamento (por exemplo, Barr et al., 2011; Grover e Ervilha, 2013). Nesta seção, comparamos a TC com o pensamento matemático, de engenharia, de design e de sistemas.

O pensamento matemático envolve a aplicação de habilidades matemáticas para resolver problemas matemáticos, como equações e funções (Sneider et al., 2014). Harel e Sowder (2005) definiram o pensamento matemático como global em muitos problemas e "... governa as formas de compreensão" (p. 31). O pensamento matemático consiste em três partes: crenças sobre matemática, processos de resolução de problemas e justificativa para soluções. O principal ponto em comum entre a TC e o pensamento matemático são os processos de resolução de problemas (Wing, 2008). A Figura 1 mostra o conjunto completo de conceitos compartilhados de pensamento computacional e matemático: resolução de problemas, modelagem, análise e interpretação de dados, e estatística e probabilidade.

A engenharia envolve habilidades necessárias para construir ou transformar coisas no mundo, a fim de construir vidas melhores (Bagiati & Evangelou, 2016), bem como "ciências e matemática aplicadas, resolver problemas e fazer coisas" (Pawley, 2009, p. 310) . O

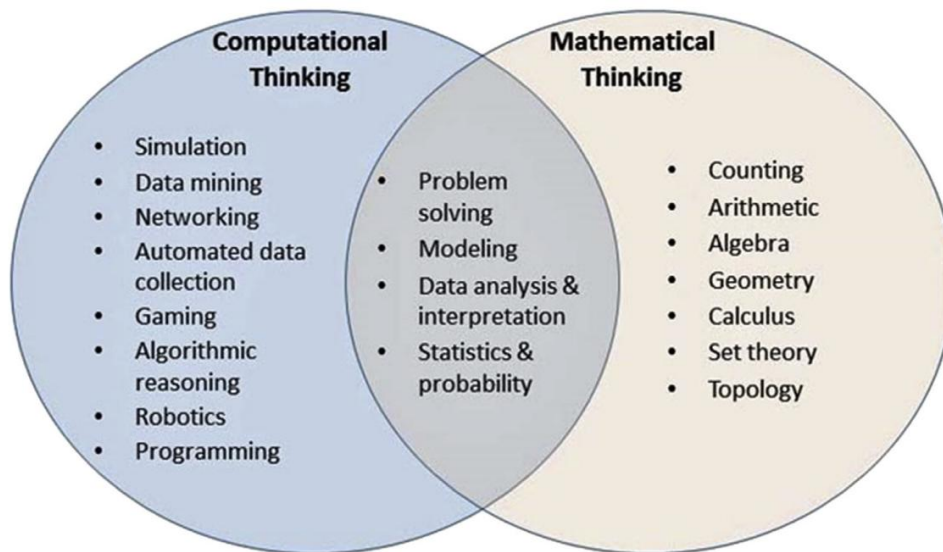


Figura 1. Semelhanças e diferenças entre PC e pensamento matemático. Adaptado de Sneider et al. (2014).

a sobreposição entre TC e engenharia inclui a resolução de problemas, juntamente com a compreensão de como sistemas complexos funcionam no mundo real (Wing, 2008). No entanto, ao contrário da engenharia, a TC pretende ajudar os humanos a compreender fenômenos complexos através de simulações e modelações, que podem transcender as restrições físicas (Wing, 2010). Para resumir, a TC, o pensamento matemático e a engenharia derivam de disciplinas diferentes. Suas diferenças residem em aplicações específicas em seu próprio domínio.

O design thinking exige que se resolva problemas pensando como designer (Razzouk & Shute, 2012). O pensamento computacional e o design thinking se concentram na resolução de problemas. O design thinking, assim como a engenharia, concentra-se na especificação do produto e nos requisitos impostos tanto pelo ser humano quanto pelo meio ambiente (ou seja, problemas práticos). Mais uma vez, a TC não está limitada por restrições físicas, permitindo às pessoas resolver problemas teóricos e práticos.

O pensamento sistêmico refere-se à capacidade de compreender várias relações entre os elementos de um determinado ambiente (Shute, Masduki, & Donmez, 2010). De acordo com o modelo de competências desenvolvido por Shute et al. (2010), pessoas com habilidades de pensamento sistêmico devem ser capazes de: (a) definir os limites de um problema/sistema, (b) modelar/simular como o sistema funciona conceitualmente, (c) representar e testar o modelo do sistema usando ferramentas computacionais e (d) tomar decisões com base no modelo.

Embora a TC e o pensamento sistêmico envolvam a compreensão e a modelagem de sistemas, a TC é mais ampla do que o pensamento sistêmico, que se concentra na identificação e compreensão do funcionamento de um sistema como um todo. CT visa resolver problemas de forma eficiente e eficaz, indo além da modelagem e compreensão para incluir design algorítmico, automação e generalização para outros sistemas/problemas. Concluindo, CT é um termo abrangente que contém design thinking e engenharia (ou seja, design de soluções eficientes), pensamento sistêmico (ou seja, compreensão e modelagem de sistemas) e pensamento matemático aplicado à solução de vários problemas.

3.1.3. Relação da TC com a ciência da computação e a programação Outra

área que precisa de esclarecimento envolve as relações entre a TC, a ciência da computação e a programação (Czerkowski & Lyman, 2015). E embora a TC tenha origem na ciência da computação (Wing, 2006), ela difere da ciência da computação porque permite que as pessoas transfiram competências de TC para outros domínios que não a programação (Berland & Wilensky, 2015).

As competências de TC não são iguais às competências de programação (Ioannidou et al., 2011), mas ser capaz de programar é um dos benefícios de ser capaz de pensar computacionalmente (Israel et al., 2015). Por exemplo, Shute (1991) examinou as relações entre competências de programação, conhecimento prévio e competências de resolução de problemas, em 260 estudantes universitários e de escolas técnicas. Os participantes que não tinham experiência anterior em programação aprenderam habilidades de programação por meio de um sistema de tutoria inteligente. Várias avaliações foram administradas para medir o conhecimento recebido dos alunos (ou seja, conhecimento de matemática e palavras), habilidades cognitivas (ou seja, memória de trabalho e velocidade de processamento de informações) e aspectos específicos das habilidades de resolução de problemas (por exemplo, identificação de problemas, sequenciamento e decomposição). Além disso, foi utilizado um teste de critério para medir as habilidades e conhecimentos de programação dos alunos após a intervenção. Os resultados de uma análise fatorial e de regressão hierárquica mostraram que a memória de trabalho, a identificação de problemas e as soluções de sequenciamento são os melhores preditores da aquisição de habilidades de programação. Assim, as competências de TC e de programação, bem como a resolução de problemas, estão intimamente relacionadas.

Em geral, o campo da ciência da computação é mais amplo do que apenas aprender sobre programação, e a TC é mais ampla do que a ciência da computação (NRC, 2010; Wing, 2006), na medida em que a TC inclui uma forma de pensar sobre atividades e problemas cotidianos. Em consonância com esta perspectiva, Lu e Fletcher (2009) propuseram que o ensino da PC não deveria sequer utilizar linguagens de programação; em vez disso, a linguagem deve ser baseada em noções familiares à maioria dos alunos para gerar a aquisição de conceitos como

abstração e algoritmos. Tal como acontece com a maioria das pesquisas direcionadas à CT, sua relação particular com a programação de computadores está evoluindo.

3.2. Intervenções para desenvolver o pensamento computacional

Os pesquisadores tentaram aproveitar ferramentas de programação, robótica, jogos/simulações e intervenções não digitais para ensinar conhecimentos e habilidades de PC em vários contextos educacionais. A população-alvo abrange desde o jardim de infância até os estudantes de graduação. A [Tabela 1](#) resume todos os estudos de pesquisa que revisamos, organizados por ferramentas de intervenção.

3.2.1. Pesquisa em CT usando ferramentas de programação

Devido à sua estreita relação com a computação e a programação, as competências em TC parecem ser melhoradas através de ferramentas computacionais, como Scratch (MIT, 2003). Quanto à equivalência de competências de programação com competências de TC, [Cetin \(2016\)](#) comparou os efeitos de empregando Scratch (grupo experimental) com linguagem C (grupo de controle) para ensinar conceitos de programação para pré-atendimento de TI professores. Este experimento durou seis semanas, e os participantes (n = 56) completaram pré e pós-testes relativos às suas realizações e atitudes em relação à programação. Além disso, nove participantes por grupo foram selecionados aleatoriamente para participar de entrevistas semiestruturadas. Os resultados mostraram que o grupo experimental teve um desempenho significativamente melhor que o controle grupo em termos de conhecimentos e habilidades de programação, mas não houve diferenças de atitude entre os grupos.

Para promover o pensamento algorítmico via Scratch, [Grover, Pea e Cooper \(2015\)](#) projetaram um CT baseado em Scratch de sete semanas curso para alunos do 7º e 8º ano (n = 54). Semelhante ao estudo [de Cetin \(2016\)](#), o ganho de CT foi medido do pré-teste ao pós-teste melhoria nas habilidades de programação. O objetivo deste quase-experimento era ver qual abordagem (instrução presencial vs. presencial e online) apoiou o aprendizado profundo em relação a conceitos computacionais, como algoritmos, loops, condicionais e decomposição. As duas condições foram equiparadas em termos de receber instrução comparável para o mesmo duração do tempo (ou seja, quatro dias por semana, 55 minutos por dia, durante sete semanas). Os resultados revelaram que ambas as abordagens levam a ganhos de CT significativamente maiores, e os alunos do grupo presencial e on-line tiveram desempenho significativamente melhor do que aqueles do grupo o grupo presencial. Além disso, ambos os grupos transferiram com sucesso os seus conhecimentos e competências de programação para programas baseados em texto. tarefas de programação.

A força do Scratch é ajudar os jovens a aprender a pensar criativamente, raciocinar sistematicamente e trabalhar de forma colaborativa, e, portanto, é adequado para facilitar a TC. É fácil de usar com seu método de programação arrastar e soltar e fornece uma experiência significativa ambiente de aprendizagem onde os alunos se envolvem em contextos específicos. Alice (Carnegie Mellon University, 1999) funciona de forma semelhante, equipado com blocos de código prontos. Comparado com o Scratch, ele se concentra na criação de projetos de programação 3D. Também pode ser utilizado para treinar CT. Por exemplo, [Denner, Werner, Campe e Ortiz \(2014\)](#) designaram aleatoriamente 320 alunos do ensino médio para um grupo de trabalho diádico ou programação individual. As habilidades de PC dos alunos foram medidas pela sua capacidade de realizar Tarefas de Alice durante o curso de um semestre. Os resultados demonstraram que os alunos que trabalham de forma colaborativa alcançaram resultados significativos pontuações de CT mais altas do que estudantes que trabalham sozinhos. E a colaboração foi especialmente benéfica para estudantes com necessidades mínimas

tabela 1

Artigos revisados classificados por ferramentas de intervenção.

Estudar	Contexto	Participantes	Intervenção		Conteúdo	Medidas
			Programa	Duração		
Cetin, 2016	Ensino superior	56 graduandos (professores em formação)	Arranhar; Linguagem C	6 semanas	Computador Ciência	Múltipla escolha; questões em aberto; pesquisas; entrevistas
Grover et al., 2015	Ensino fundamental	54 Scratch para alunos do 7º e 8º ano		7 semanas	Curso de TC	Múltipla escolha; questionários; atribuições; baseado em texto codificação; projetos
Denner et al., 2014	Ensino Médio	320 alunos	Alice (solo vs. pares)	Pesquisas de Programação de um semestre; tarefas		
Werner et al., 2012	Ensino Médio	311 alunos	Alice (solo vs. pares)	Pesquisas de Programação de um semestre; tarefas		
Atmatzidou & Demetriadi, 2016	Ensino médio	164 alunos	Lego Mindstorms Robótica	Um ano	Curso de TC	Questionários; pensar alto; entrevistas
Berland & Wilensky, 2015	Ensino fundamental	78 alunos da 8ª série	Lego versus virtual robótica	5 dias	Ciência	Arquivos de log; testes; questionários
Basu et al. (2017)	Ensino fundamental	98 alunos do 6º ano	Plataforma CTSIM	13 dias	Ecologia, questões de CT; explicações;	construindo algoritmos
Bers et al., 2014	Jardim da infância Ensino fundamental	53 alunos 51 alunos	TangibleK Robótica 20h Foto; Pequeno Básico 3h		Curso de TC Poema	Questionários Preencha os espaços em branco; questões;
Kim et al., 2013	Ensino superior	110 alunos do segundo ano (pré-atendimento professores do ensino fundamental)	PPS	15 semanas	Computador Ciência	Múltipla escolha; pesquisas
Yadav et al., 2014	Ensino superior	141 alunos	Módulo CT	Uma semana	Educacional psicologia	Questionários

experiência em programação. Estas descobertas são consistentes com as relatadas numa experiência anterior conduzida por [Werner, Denner, Campe e Kawamoto \(2012\)](#) testando 311 estudantes do ensino secundário.

[Basu, Biswas e Kinnebrew \(2017\)](#) consideraram de forma semelhante os construtos CT como conceitos relacionados à programação, como sequenciamento, loops e variáveis; e consideraram iteração, decomposição de problemas, abstração e depuração como práticas de TC. Eles projetaram a plataforma CTSiM para integrar a aprendizagem de ecologia e TC para alunos do 6º ano. Basu et al. empregou um design de pré-teste/pós-teste para testar a eficácia do andaime fornecido por um agente virtual incorporado no CTSiM. Tanto o grupo experimental ($n = 52$) quanto o grupo de controle ($n = 46$) aprenderam TC e ecologia via CTSiM, mas apenas o grupo experimental recebeu andaimes. Ou seja, quando os alunos falharam em uma determinada tarefa 3e5 vezes, o andaime foi acionado. Nesse caso, o agente virtual fornecia prompts de conversa e os alunos respondiam escolhendo uma das opções, o que por sua vez acionava uma resposta do agente. As conversas entre agentes e alunos foram pré-programadas para ajudar os alunos com dificuldades. No grupo de controle, tal funcionalidade foi desabilitada. Os testes de ecologia exigiam que os alunos escolhessem as respostas corretas e justificassem suas respostas. Os testes CT exigiam que os alunos previssem resultados após lerem segmentos do programa e construísssem algoritmos com construções CT para modelar cenários. Os pré e pós-testes mostraram ganhos significativos em ambos os grupos em ecologia e TC. E quando as pontuações do pré-teste foram usadas como covariável, o grupo experimental (com andaimes) superou significativamente o grupo de controle em ganhos de aprendizagem ecológica e habilidades de TC, com tamanhos de efeito de 0,36 e 0,31, respectivamente.

3.2.2. Pesquisa usando robótica

Outra área frutífera para desenvolver competências de TC é a robótica, que também está intimamente relacionada com a programação. A robótica Lego é particularmente popular. Por exemplo, Lego Mindstorms (<http://www.lego.com/en-us/mindstorms>) foi usado para melhorar as habilidades de PC de alunos do ensino médio ($n = 164$) durante um ano ([Atmatzidou & Demetriadi, 2016](#)). As 44 sessões de duas horas tiveram como foco o desenvolvimento das seguintes habilidades de PC: decomposição de problemas, abstração de informações essenciais, generalização da solução para diversos problemas, criação de algoritmos e automatização de procedimentos.

A resolução de problemas de programação de robôs revelou as competências de TC dos alunos, que foram medidas através de rubricas relacionadas com a qualidade do desempenho na resolução de problemas. Os dados quantitativos mostraram que todos os participantes, independentemente da idade ou sexo, melhoraram de forma semelhante nas suas competências de PC após a intervenção. Os dados qualitativos gerados a partir de entrevistas e protocolos de pensamento em voz alta confirmaram a eficácia da robótica para desenvolver conceitos de TC e resolver problemas de forma eficaz.

A robótica Lego é uma atividade física. [Berland e Wilensky \(2015\)](#) compararam os efeitos da robótica Lego ($n = 34$) versus robótica virtual ($n = 44$) entre alunos do 8º ano. Pré e pós-testes em programação de pseudocódigo mediram ganhos de CT. Ambos os grupos de estudantes melhoraram significativamente suas habilidades de TC e não houve diferença significativa pós-teste entre os dois grupos. É interessante notar que os dois grupos perceberam os problemas de forma diferente. Ou seja, o grupo de robótica virtual tendeu a perceber o problema como um todo e depois a tentar decompor o problema até aos seus detalhes; enquanto o grupo de robótica física inicialmente se concentrou nas partes constituintes.

Quão cedo as crianças podem aprender habilidades de TC? Um estudo analisou o ensino de competências de PC a alunos do jardim de infância ($n = 53$) ([Bers et al., 2014](#)). Esses pesquisadores desenvolveram o currículo TangibleK Robotics (<http://ase.tufts.edu/DevTech/tangiblek/>) que incluiu 20 horas de instrução e um projeto final para medir o desenvolvimento de TC dos alunos em termos de depuração, sequenciamento, loops e condicionais. No entanto, medidas repetidas entre tarefas não revelaram qualquer desenvolvimento linear do PC em alunos do jardim de infância. Os investigadores especularam que ou (a) os alunos do jardim de infância são simplesmente demasiado jovens para desenvolver competências de TC, ou (b) a intervenção robótica foi demasiado difícil para eles ([Bers et al., 2014](#)). Portanto, é importante ensinar os conceitos de PC de uma forma adequada às fases de desenvolvimento dos alunos.

3.2.3. Pesquisa utilizando design de jogos e outras ferramentas de intervenção

AgentSheets é uma ferramenta de autoria que utiliza design de jogos para ensinar PC, bem como para promover o interesse e as habilidades dos alunos em ciência da computação (ver [Ioannidou et al., 2011](#) para detalhes). AgentSheets fornece um nível de entrada baixo para alunos inexperientes, mas não impede que alunos com habilidades avançadas criem sistemas de jogos complexos. Em um estudo, professores e estudantes de faculdades comunitárias aprenderam em dois workshops de verão como animar interações entre objetos por meio de programação, usando cinco tarefas de design de AgentSheets ([Basawapatna, Koh, Repenning, Webb, & Marshall, 2011](#)). Ao contrário de outros estudos, esta pesquisa mediu a TC com base no conhecimento dos alunos sobre padrões relevantes, definidos como interações de objetos, como colisão e absorção. Esses padrões CT representam a interseção de jogos e simulações científicas. Após duas semanas de intervenção, os autores testaram as habilidades de TC dos participantes com oito perguntas, cada uma representando um fenômeno que compartilhava padrões de TC nos cinco jogos (por exemplo, colisão de trenó). Os participantes precisavam identificar e justificar o uso dos padrões CT para criar simulações dos fenômenos. Geralmente, todos os participantes se saíram muito bem na identificação correta dos padrões de TC, conforme indicado pela porcentagem correta de suas respostas. No entanto, os autores observaram que os resultados não significam necessariamente que os alunos possam transferir conhecimentos de PC desde a concepção de jogos para simulações de fenômenos científicos.

O ensino de PC não requer necessariamente ferramentas digitais. Uma estratégia de programação de papel e lápis (PPS) foi criada para cursos não relacionados à ciência da computação ($n = 110$) ([Kim, Kim, & Kim, 2013](#)). Permitiu que os participantes ($n = 55$) de um grupo escolhessem a forma preferida de visualizar e representar ideias de programação (por exemplo, através de mapas conceituais, tabelas, palavras ou símbolos). Seus efeitos foram comparados a outro grupo, que se envolveu em um curso típico ($n = 55$) durante 15 semanas. A instrução típica neste caso refere-se ao curso regular de programação, utilizando LOGO (Softtronics, Inc., 2002). Os pré e pós-testes revelaram que os alunos que utilizaram a estratégia de programação com papel e lápis demonstraram uma compreensão significativamente melhor da TC e mais interesse na UC do que aqueles que aprenderam através do ensino típico do curso. Os autores argumentaram, portanto, que o ensino da TC sem computadores pode ser eficaz e eficiente para estudantes que não sejam ciências da computação.

Um módulo instrucional de TC baseado em palestras também poderia mudar a compreensão e as atitudes dos professores em formação em relação à integração de TC nas suas salas de aula (Yadav, Mayfield, Zhou, Hambrusch, & Korb, 2014). Os pesquisadores testaram um módulo de 100 minutos focado em conceitos de CT como decomposição, abstração, algoritmos e depuração. Após o treinamento, os participantes preencheram questionários com questões abertas relacionadas ao PC e suas conexões com diversas disciplinas. Em comparação com o grupo de controle sem tratamento ($n = 154$), uma porcentagem maior daqueles no grupo experimental ($n = 141$) que receberam o treinamento em TC consideraram a TC uma forma de pensar para resolver problemas, com ou sem a ajuda de computadores. Além disso, perceberam que a TC não se restringia ao campo da informática. Por outro lado, o grupo de controle via a TC de forma restrita como o uso de computadores para completar tarefas computacionais específicas e restrito a assuntos que envolvem computadores, como ciência da computação e matemática.

3.2.4. Habilidades de TC subjacentes à programação, robótica e design de jogos

A justificativa para usar programação, robótica e design de jogos para melhorar as habilidades de TC é porque cada uma dessas áreas enfatiza vários componentes de TC. Por exemplo, escrever programas de computador requer a análise do problema (por exemplo, determinar o objectivo a atingir) e depois dividir o problema nos seus processos constituintes (por exemplo, identificar sub-objectivos e passos associados para atingir o objectivo). Escrever programas eficientes requer abstração e generalização. Por exemplo, se uma etapa precisa ser repetida quatro vezes, uma solução eficiente envolve repetir a etapa em vez de escrever o mesmo código quatro vezes. Além disso, parte do código de programação pode ser reutilizada em problemas semelhantes, com pequenos ajustes, em vez de reescrever um novo programa do zero. E para testar a correção e eficiência de um programa, é necessária depuração. É por isso que a programação é frequentemente utilizada para promover competências de PC.

Da mesma forma, a robótica proporciona aos alunos experiências tácteis para resolver problemas através de competências de TC. Os alunos precisam identificar o problema/objectivo geral do robô e, em seguida, decompor o problema (por exemplo, descobrir o número de etapas ou subobjetivos para atingir o objetivo). Para esse fim, os alunos desenvolvem algoritmos para o robô, para que ele possa seguir as instruções e agir de acordo. Quando o robô não age conforme o esperado, a depuração entra em ação. A depuração requer processos iterativos de testes e modificações sistemáticos.

Finalmente, assim como acontece com a programação e a robótica, o design e a jogabilidade do jogo envolvem vários objetivos a serem resolvidos pelos jogadores. Podem ser objetivos menores representados nos níveis do jogo e objetivos maiores representados por níveis de chefe e/ou parte da narrativa. Para ter sucesso, os jogadores precisam elaborar planos de solução e, se um plano falhar, um plano modificado será desenvolvido. Ao testar sistematicamente vários planos, os jogadores encontram a estratégia mais eficaz para superar os desafios do jogo. Além disso, jogadores habilidosos são capazes de adotar estratégias utilizadas anteriormente para resolver novos problemas. Assim, decomposição de problemas, teste e depuração sistêmica, generalização e iteração são habilidades exigidas em jogos e são componentes importantes da CT.

3.3. Avaliação do pensamento computacional

Devido à variedade de definições e conceptualizações da TC, não é surpreendente que a avaliação precisa da TC continue a ser uma grande fraqueza nesta área. Atualmente não existe uma avaliação amplamente aceita da TC. Isto torna difícil medir a eficácia das intervenções de forma fiável e válida (Grover & Pea, 2013; Kim et al., 2013; Settle et al., 2012). Não ter uma avaliação de TC padrão também torna muito difícil comparar os resultados entre vários estudos de TC. Os investigadores tendem a desenvolver e aplicar as suas próprias medidas de TC em vários estudos, dependendo da sua operacionalização específica da TC (Kim et al., 2013).

Os questionários e inquéritos são as medidas mais utilizadas para avaliar o conhecimento e/ou as atitudes em relação ao PC (por exemplo, Atmatzidou & Demetriadis, 2016; Denner et al., 2014; Yadav et al., 2014). Por exemplo, Cetin (2016) utilizou inquéritos para medir a compreensão conceptual dos futuros professores sobre construções de programação, como arrays e loops, bem como as atitudes em relação à ciência da computação. Yadav et al. (2014) utilizaram de forma semelhante inquéritos para examinar a compreensão dos professores em formação sobre PC e as suas atitudes relativamente à integração do PC no seu ensino no futuro. Kim et al. (2013) elaboraram itens de questionário para medir especificamente o pensamento lógico dos alunos do segundo ano da faculdade, bem como a compreensão conceitual e o interesse pela ciência da computação. Vendo a TC como o núcleo da alfabetização computacional, Jun, Han, Kim e Lee (2014) aplicaram questionários sobre alfabetização computacional entre alunos do ensino fundamental.

Outros investigadores realizaram entrevistas e observações com os participantes para compreender o desenvolvimento de competências de PC (por exemplo, Cetin, 2016; Israel et al., 2015). Alguns dos investigadores avaliaram as competências de PC através de projetos e tarefas relacionadas. Por exemplo, Denner et al. (2014) e Werner et al. (2012) desenvolveram uma avaliação baseada no desempenho para medir o PC, permitindo que alunos do ensino médio concluíssem projetos, como uma tarefa de três estágios desenvolvida com Alice. Os alunos precisavam resolver problemas específicos (por exemplo, salvar uma fada de uma floresta mágica). Outros investigadores testaram os ganhos de aprendizagem do conhecimento da área disciplinar, após incorporarem a PC no ensino disciplinar (por exemplo, Cetin, 2016; Sengupta et al., 2013).

3.3.1. Avaliações baseadas em scratch

Vários grupos de pesquisadores usaram o Scratch para desenvolver avaliações sistemáticas de TC. Brennan e Resnick (2012) afirmaram que os projetos baseados no Scratch poderiam apoiar a aprendizagem a longo prazo, tornando a aprendizagem significativa num contexto específico. Eles também defenderam uma medida dinâmica para avaliar as habilidades de TC dos usuários do Scratch (8 a 17 anos) ao longo do tempo, revelando a progressão da aprendizagem e medindo tanto a compreensão conceitual quanto a aplicação das habilidades de TC. A avaliação do CT incluiu análise formativa, entrevistas e projetos de design. A análise formativa envolveu a avaliação dos portfólios dos usuários para ver o desenvolvimento da CT através de projetos concluídos ao longo do tempo. As entrevistas permitiram que os pesquisadores se aprofundassem nos processos de pensamento dos usuários. Houve também três conjuntos de projetos de design, de dificuldade variada, que foram empregados como

medidas de desempenho. Cada conjunto incluía duas tarefas do mesmo nível de dificuldade, mas com representações diferentes. Os usuários tiveram que escolher uma tarefa de cada conjunto para descrever e depurar.

Para se alinhar com um curso específico de CT baseado no Scratch para alunos do ensino médio, [Grover et al. \(2015\)](#) conceberam um sistema de avaliações formativas e sumativas para apoiar e medir competências de PC. Os itens e questionários de múltipla escolha ao longo do curso assemelhavam-se ao pseudocódigo do Scratch. Os itens foram utilizados para consolidar o conhecimento dos alunos sobre conceitos de programação, com feedback oportuno servindo para explicar diversas ideias e procedimentos. As tarefas do curso foram concluídas usando Scratch e avaliadas por rubricas específicas. A medida somativa foi um teste final com itens de múltipla escolha e questões abertas, juntamente com um projeto final realizado com um parceiro. Além disso, os pesquisadores desenvolveram um teste baseado em exames de programação AP para testar se os alunos poderiam transferir o que aprenderam para um código de programação baseado em texto real.

Outra avaliação baseada no Scratch é chamada PECT (progressão do pensamento computacional inicial), projetada para alunos do ensino fundamental ([Seiter & Foreman, 2013](#)). Isso representa uma diretriz para medir três facetas da TC. A primeira faceta incluiu variáveis de evidência, que foram coletadas diretamente das categorias de comandos embutidas no Scratch (por exemplo, operadores e condicionais). A segunda faceta foram as variáveis de padrão (por exemplo, movimento e interatividade), que são mais amplas do que as evidências específicas da biblioteca de pseudocódigos do Scratch. A última faceta foram os conceitos de CT, como decomposição, abstração e algoritmos. Os níveis de proficiência dos alunos (ou seja, básico, em desenvolvimento e proficiente) por faceta foram medidos de acordo com as rubricas.

3.3.2. Avaliação baseada em jogos/simulação

Outros pesquisadores propuseram avaliar a TC por meio da análise de padrões de TC (CTPA) em escolas de ensino fundamental e médio. Os padrões CT são conceitos abstratos de programação relacionados a interações de objetos que os alunos usam para desenvolver jogos e simulações ([Basawaputra et al., 2011](#); [Ioannidou et al., 2011](#)). Construir interações de objetos em jogos exigiu 12 padrões CT, definidos por [Ioannidou et al. \(2011\)](#). [Repenning et al. \(2015\)](#) posteriormente refinaram esses 12 padrões em nove aspectos, para medir a transferência de CT dos jogos para a modelagem de fenômenos científicos em cursos STEM. O CTPA é usado para gerar relatórios gráficos semelhantes a teias de aranha dos nove padrões, comparando os produtos dos alunos com exemplares. Esse tipo de relatório fornece informações visuais sobre a natureza e o grau das habilidades dos alunos e quais aspectos precisam ser melhorados. Os relatórios ao longo do tempo podem demonstrar a progressão das habilidades de TC dos alunos. Assim, os professores poderiam fornecer apoio instrucional tendo acesso a tais relatórios gráficos.

3.3.3. Balanços CT validadas para uso genérico

As avaliações acima mencionadas foram elaboradas exclusivamente para fins de um estudo específico baseado em uma intervenção específica. Recentemente, pesquisadores desenvolveram uma escala para medir a TC ([Rom an-Gonzalez, Perez-Gonzalez, & Jimenez-Fernandez, 2017](#)). Esta escala inclui 28 itens e leva cerca de 45 minutos para ser concluída. Ele se concentra em conceitos computacionais, como direções e sequências, loops, condicionais if e funções simples. A confiabilidade desta escala é de 0,79. Os autores testaram sua validade entre 1.251 estudantes espanhóis do 5º ao 10º ano. Os resultados mostraram correlações significativas com outros testes padronizados sobre habilidades mentais e habilidades de resolução de problemas. O teste de resolução de problemas explicou 44,7% da variância da escala CT, enquanto o teste de capacidade mental explicou 27%. Assim, os autores concluíram que o PC se assemelha mais às habilidades de resolução de problemas e menos alinhado às habilidades de raciocínio, espaciais e verbais.

Esforços semelhantes foram conduzidos e relatados por [Korkmaz, Çakir e Ozden \(2017\)](#). Os pesquisadores desenvolveram uma escala CT composta por 29 itens, medindo cinco fatores (ou seja, criatividade, cooperação, pensamento algorítmico, pensamento crítico e resolução de problemas). Eles primeiro conduziram análise fatorial exploratória (n = 726) e depois análise fatorial confirmatória (n = 580) entre estudantes universitários turcos. Os resultados mostraram bons índices de ajuste de dados e evidências de que os itens mediram os construtos alvo. As confiabilidades para os cinco fatores variaram de 0,73 a 0,87. Esta escala mede o TC de forma diferente da anterior porque os investigadores operacionalizaram o TC de forma diferente, com base no seu próprio quadro de TC (descrito na Seção 3.4.).

Em conclusão, esta revisão das avaliações de PC demonstra a necessidade premente de uma avaliação fiável e válida das competências de PC em todos os contextos educativos, para que os investigadores possam determinar se as suas intervenções de PC são eficazes ou não, em vez de apenas olharem para os ganhos de aprendizagem ou para as mudanças de atitude através de medidas caseiras. A avaliação do PC também exige informação dinâmica que possa refletir as capacidades e a progressão dos alunos ao longo do tempo.

3.4. Modelos de pensamento computacional

Depois de revistos os componentes, intervenções e avaliações da TC, esta secção examina os quadros teóricos da TC. Tal como acontece com as definições e avaliações de CT, não existem modelos ou estruturas acordadas para a CT ([Atmatzidou & Demetriadis, 2016](#)). Nesta secção, examinamos quatro modelos de TC propostos por pesquisadores.

Primeiro, [Atmatzidou e Demetriadis \(2016\)](#) apresentaram um modelo de TC simples e descritivo, baseado na operacionalização da TC em estudos anteriores. O modelo consiste em cinco facetas: abstração, generalização, algoritmos, modularidade e decomposição. Também fornece exemplos de comportamentos que os alunos devem demonstrar como evidência para cada faceta (ver [Atmatzidou & Demetriadis, 2016](#), p. 664 para detalhes). Resumidamente, abstração significa destilar os padrões centrais de sistemas complicados; a generalização envolve a aplicação de estratégias de resolução de problemas a diferentes contextos; algoritmos referem-se a etapas/instruções ordenadas para implementar soluções; modularidade significa a automação de soluções de resolução de problemas; e a decomposição implica a quebra de sistemas/coisas complexas em pedaços gerenciáveis.

O segundo modelo consiste em conceitos e habilidades de PC que devem ser incorporados em cursos de ensino fundamental e médio, como matemática, ciências, estudos sociais e artes da linguagem (ver [Barr & Stephenson, 2011](#), p. 52). Ele define as principais facetas da TC (ou seja, coleta de dados,

análise, representação de dados, decomposição, abstração, algoritmos, automação, paralelismo e simulação) em várias disciplinas. Estas facetas do PC devem ter representações diferentes em diferentes disciplinas (por exemplo, representar dados com gráficos ou tabelas em matemática e representar padrões linguísticos em artes linguísticas). No entanto, as demonstrações específicas das facetas da TC em disciplinas específicas não estão claramente indicadas no artigo. Além disso, os exemplos de práticas de ensino fornecidos são demasiado vagos para que os professores os possam realmente empregar. Por exemplo, a abstração nas aulas de ciências é descrita apenas como modelagem de uma entidade física. A fraqueza deste modelo decorre de (a) nenhuma definição clara por faceta de TC, tornando a operacionalização muito difícil, e (b) falha em distinguir conceitos de habilidades (por exemplo, a abstração é tanto um conceito quanto uma habilidade). Assim, este modelo pode ser melhorado antes de servir como diretriz para professores do ensino fundamental e médio.

Brennan e Resnick (2012) apresentaram uma estrutura de TC no contexto do uso do Scratch para facilitar a TC. Eles categorizam-dividiu a CT em três áreas: conceitos, práticas e perspectivas. A Tabela 2 mostra nosso resumo de sua estrutura.

Um artigo de revisão sobre a integração da TC em ambientes de ensino fundamental e médio, elaborado por Lye e Koh (2014), baseou-se na estrutura de Brennan e Resnick. Mais tarde, Zhong, Wang, Chen e Li (2016) revisaram esse modelo (ver página 565 para detalhes) adicionando instruções aos conceitos de TC e iteração às práticas de TC. Eles também reformularam as perspectivas da TC para enfatizar a criatividade e a colaboração. No entanto, não detalharam essas modificações, o que dificulta a interpretação do modelo revisado.

Um quarto modelo recente visa fundir a TC e o ensino regular em sala de aula (Weintrop et al., 2016). Os pesquisadores analisaram 34 planos de aula para cursos de matemática e ciências do ensino médio, codificando práticas de ensino relacionadas às facetas da TC. Em seguida, categorizaram as facetas específicas em práticas mais amplas de TC e refinaram a categorização após consultarem os criadores de planos de aula, professores do ensino secundário em serviço e especialistas em TC e na concepção de currículos. Finalmente, chegaram a uma taxonomia contendo quatro categorias de TC com 22 práticas de TC (ver Tabela 3). A sua taxonomia baseia-se em atividades específicas em sala de aula e apresenta exemplos concretos de atividades em sala de aula de PC, mostrando como os planos de aula podem ser concebidos seguindo a taxonomia. Este modelo é adaptado aos cursos STEM no ensino secundário e mostra-se promissor no que diz respeito à integração da TC no ensino secundário. No entanto, mais pesquisas são necessárias para validar este modelo.

A falta de um modelo consistente pode causar problemas na concepção de intervenções de apoio à aprendizagem de PC e na avaliação de conhecimentos e competências de PC em vários contextos educativos. Isto leva à próxima parte deste artigo; nomeadamente, o nosso modelo de competências de TC proposto, que pode ser utilizado para orientar a avaliação e o apoio às competências de TC.

4. Nossa definição e estrutura de pensamento computacional

Com base nas definições e modelos acima mencionados, definimos TC como a base conceptual necessária para resolver problemas de forma eficaz e eficiente (isto é, algoritmicamente, com ou sem a ajuda de computadores) com soluções que são reutilizáveis em diferentes contextos. Conforme afirmado anteriormente, os componentes da TC que surgem com mais frequência na literatura são: abstração, decomposição, algoritmos e depuração. Nosso modelo tenta compreender os processos cognitivos subjacentes a cada uma dessas facetas da TC e os comportamentos associados que podem nos ajudar a desenvolver um modelo de competências que possa ser usado para a avaliação da TC. Além disso, identificamos a iteração e a generalização como mais duas competências importantes no desenvolvimento da PC.

Nosso modelo de CT enfatiza a importância de abordar os problemas de forma sistemática. A decomposição de problemas envolve dividir um problema complexo em partes menores e usar processos sistemáticos para resolver cada um desses problemas menores. A depuração iterativa e sistemática garante que cada parte dos problemas menores seja resolvida de forma eficiente e sem pontas soltas. Abstração é o ato de encontrar padrões em problemas e soluções e, assim, estar em posição de generalizar soluções para conjuntos de problemas semelhantes. Finalmente, o design de algoritmos permite o desenvolvimento de ferramentas/procedimentos reutilizáveis para resolver classes de problemas.

Com base em pesquisas anteriores, categorizamos a CT em seis facetas principais: decomposição, abstração, algoritmos, depuração, iteração e generalização. As seis facetas do nosso modelo CT estão descritas na Tabela 4.

Este enquadramento da TC sustenta, e pode ser evidenciado através de, uma variedade de áreas temáticas do ensino fundamental e médio, incluindo Matemática, Ciências e até mesmo artes da língua inglesa.

Tabela 2
Resumo da estrutura de TC proposta por Brennan e Resnick (2012).

Conceitos de TC	Sequências: instruções para o computador executar comportamentos Loops: repete a mesma instrução por um determinado número de vezes Paralelismo: Concorrência de múltiplas instruções Eventos: gatilhos para que certas ações aconteçam para criar ambientes interativos Conacionais: Restrições na execução de instruções, permitindo diferentes resultados Operadores: operações matemáticas e de strings Dados: armazenamento, recuperação e atualização de dados
Práticas de TC	Ser incremental e iterativo: Processos iterativos para projetar e implementar soluções, passo a passo Teste e depuração: Processos de tentativa e erro para testar e remover defeitos conforme garantido Reutilizar e remixar: Construindo instruções reutilizáveis; construir novos produtos com base no trabalho de outros Abstração e modularidade: Modelagem de sistemas complexos com elementos básicos
Perspectivas da TC	Expressando: Percepção da computação como forma de expressão e criação Conectando: Percepção da computação como forma de interagir e trabalhar com outras pessoas Questionamento: Levantar questões e usar a tecnologia para resolver problemas da vida real

Tabela 3

Taxonomia de CT em cursos STEM, adaptada de [Weintrop et al. \(2016\)](#).

Categoria CT	Política de tecnologia computacional	Definição
Práticas de dados	Coleção de dados	Colete dados usando várias ferramentas computacionais
	Criação de dados	Gere dados para sistemas grandes e complexos
	Manipulação de dados	Reorganize os dados de maneira significativa
	Análise de dados	Use ferramentas computacionais para analisar dados e tirar conclusões válidas
	Visualização de dados	Comunique e apresente dados de diversas maneiras
Modelagem e Simulação	Compreensão conceitual	Entenda profundamente os conceitos por meio da modelagem
	Testando soluções	Resolva problemas por meio de testes de hipóteses
	Avaliação do modelo	Avalie a eficácia dos modelos
	Projeto do modelo	Selecione elementos essenciais para modelos
	Construção de modelo	Implementar novos modelos ou ampliar modelos existentes
Resolução de problemas computacionais	Preparação da solução	Decompor e reformular problemas usando ferramentas computacionais adequadas
	Programação	Possuir conhecimentos e habilidades básicas de programação
	Seleção de ferramentas	Avalie prós e contras de ferramentas computacionais plausíveis
	Avaliação da solução	Avalie prós e contras de possíveis soluções
	Desenvolvimento de solução	Desenvolver soluções que possam ser aplicadas a uma ampla gama de problemas
	Abstração	Destilar as informações mais relevantes de um problema
Sistemas a pensar	Depuração	Identificar e corrigir erros
	Investigação do sistema	Entenda as funções do sistema como um todo
	Compreendendo os relacionamentos	Compreender o funcionamento e a inter-relação dos elementos de um sistema
	Pensamento multicamadas	Pense a partir de múltiplas perspectivas e níveis
	Comunicação	Transmitir informações de forma eficaz e eficiente
	Administração de sistema	Defina o escopo dos sistemas e gerencie a complexidade

4.1. Comparação com outros modelos

Nosso modelo se concentra na base conceitual subjacente necessária para abordar problemas através de uma perspectiva de TC e como esse tipo de perspectiva de TC pode ser destacada e apoiada nas disciplinas atuais do ensino fundamental e médio. Isto contrasta com os modelos que se concentram em apenas uma área temática específica, como [Brennan e Resnick \(2012\)](#) que limitam seu modelo de PC a conceitos relacionados a codificação. As três facetas do seu modelo são hierárquicas com a base composta por conceitos de programação, como loops e condicionais. A próxima camada refere-se às práticas de computação baseadas no conhecimento conceitual, enquanto a camada mais alta camada representa perspectivas sobre a computação (por exemplo, uma forma de expressão, criação e comunicação).

O modelo de 2016 de [Weintrop et al.](#) é derivado de práticas de ensino do ensino médio que ocorrem em cursos STEM. As quatro facetas em seu modelo representa uma coleção de possíveis atividades de sala de aula a partir das quais os professores podem selecionar as mais relevantes para facilitar a aquisição de TC. Assim, o seu modelo é restrito aos cursos STEM do ensino médio, onde sentimos que um A base para a TC começa muito antes, de forma análoga à investigação científica ou ao raciocínio matemático.

4.2. Exemplos de modelos emergentes de K-12 CT

Nosso modelo de conhecimentos e habilidades de TC baseado em competências se esforça para informar métodos de avaliação para medir os conhecimentos básicos de TC em alunos do ensino fundamental e médio. Vários grupos de pesquisa estão apenas começando tentativas de medir a TC no ensino fundamental e médio configurações. Apresentamos aqui vários exemplos, terminando com uma ilustração do trabalho dos coautores relacionado à avaliação sobre o fundamentação conceitual de diversas facetas importantes da TC; nomeadamente decomposição de problemas, abstração e design de algoritmos.

Conforme observado anteriormente, vários grupos de pesquisa estão estudando a codificação dos alunos no Scratch. Uma ferramenta chamada Dr. Scratch (<http://www.drscratch.org/>) está atualmente em teste Beta e é usado como base para analisar o código Scratch quantificando o uso de loops, sequências e outras facetas lógicas da codificação descritas por [Brennan e Resnick \(2012\)](#).

Pesquisadores do SRI ([Grover et al., 2015](#)) projetaram e implementaram um currículo para o ensino médio chamado Foundations for Avançando o Pensamento Computacional (FACT). A FACT se esforça para obter uma visão abrangente das habilidades de TC dos alunos, examinando variáveis cognitivas, interpessoais e intrapessoais. FACT utiliza estratégias pedagógicas para apoiar a transferência de blocos para programação baseada em texto, juntamente com avaliações formativas e sumativas (incluindo questionários e testes, bem como tarefas de programação abertas) relacionadas à aquisição de habilidades de pensamento computacional. Suas descobertas mostram que alunos de 11 a 14 anos que usam o currículo FACT experimentam melhor aprendizado algorítmico, compreensão de computação e transferência de habilidades do Scratch para um contexto de programação baseado em texto. Com base nesta pesquisa, [Grover \(2017; ver <http://stemforall2017.videohall.com/presentations/872>\)](#) sugere um enquadramento de variáveis, expressões, loops e algoritmos (VELA) para preparar jovens alunos para a CT.

A pesquisa em andamento pela EdGE no TERC2 envolve o estudo do desenvolvimento do CT dentro do jogo lógico Zoombinis. O EdGE estuda como o comportamento baseado em jogos pode prever o conhecimento implícito e ajudar os professores a apoiar a aprendizagem STEM explícita em

² TERC é uma organização sem fins lucrativos que estuda inovações em educação matemática e científica desde 1966. Em 2009, o grupo Educational Gaming Environments (EdGE) foi fundada para estudar a aprendizagem STEM em jogos digitais.

Tabela 4

Resumo de nossas facetas de TC e suas definições.

Faceta	Definição
Decomposição	Dissecar um problema/sistema complexo em partes gerenciáveis. As partes divididas não são peças aleatórias, mas elementos funcionais que compreendem coletivamente todo o sistema/problema.
Abstração	Extraia a essência de um sistema (complexo). Abstração tem três subcategorias: (a) Coleta e análise de dados: Colete as informações mais relevantes e importantes de múltiplas fontes e compreenda o relacionamentos entre conjuntos de dados multicamadas; (b) Reconhecimento de padrões: Identificar padrões/regras subjacentes à estrutura de dados/informação; (c) Modelagem: Construir modelos ou simulações para representar como um sistema opera e/ou como um sistema funcionará no futuro.
Algoritmos	Projete instruções lógicas e ordenadas para apresentar uma solução para um problema. As instruções podem ser executadas por um ser humano ou computador. Existem quatro subcategorias: (a) Projeto de algoritmo: Crie uma série de etapas ordenadas para resolver um problema; (b) Paralelismo: Realizar um certo número de etapas ao mesmo tempo; (c) Eficiência: Projetar o menor número de etapas para resolver um problema, eliminando etapas redundantes e desnecessárias; (d) Automação: Automatizar a execução do procedimento quando necessário para resolver problemas semelhantes.
Depuração	Detecte e identifique erros e, em seguida, corrija-os quando uma solução não funcionar como deveria.
Iteração	Repetir processos de design para refinar soluções, até alcançar o resultado ideal.
Generalização	Transferir competências de TC para uma ampla gama de situações/domínios para resolver problemas de forma eficaz e eficiente.

a sala de aula (Rowe, Asbell-Clarke, & Baker, 2015). Conhecimento implícito refere-se ao conhecimento que ainda não pode ser formalizado ou expressa por um aluno, mas pode ser evidente através de ações e comportamentos.

No Zoombinis, os jogadores envolvem-se em muitos ciclos de desenvolvimento de estratégias de CT que podem ser evidenciados pelos seus padrões de atividade. No jogo, os jogadores são desafiados a classificar Zoombinis de acordo com regras lógicas subjacentes (por exemplo, o quebra-cabeça chamado Alérgicos Cliffs rejeitarão Zoombinis com certas características, ou uma combinação de características em níveis superiores (ver Fig. 2).

Os jogadores normalmente passam de tentativa e erro (testando Zoombinis arbitrários) para um padrão de teste sistemático, onde usam estratégias de resolução de problemas, algumas das quais os investigadores podem identificar como evidência de PC. Por exemplo, os jogadores podem consistentemente testar se Zoombinis com uma ou mais características comuns (por exemplo, aqueles com óculos) ajudam o aluno a identificar um padrão para resolver o problema



Figura 2. Captura de tela do nível "Penhascos Alérgicos" no Zoombinis.

quebra-cabeça. Esta mudança, de tentativa e erro para testes sistemáticos, fornece evidências da decomposição do problema. À medida que os jogadores acumulam evidências, eles podem ver padrões comuns e começar a testar uma solução parcial, eventualmente demonstrando sua capacidade de abstrair os padrões em uma regra subjacente do quebra-cabeça. Jogadores bem-sucedidos exibem estratégias repetidas, ou algoritmos, implementados em uma coleção de quebra-cabeças. Por exemplo, uma vez que os jogadores adotem a estratégia de tentar uma variável de cada vez e, em seguida, implementar peças singulares de informação numa solução final, esse padrão pode ser observado em vários puzzles (e outras situações de resolução de problemas).

Para validar a aprendizagem dos alunos destas facetas da TC em função de jogar Zoombinis, a EdGE e a Empirical Games conceberam um conjunto de itens de avaliação online que se concentram em várias das facetas fundamentais da TC: decomposição de problemas, abstração e conceção de algoritmos. Os pesquisadores estão atualmente concluindo um estudo de validação com 300 alunos do ensino fundamental e médio. As avaliações eram conceitual e estruturalmente iguais para alunos do ensino fundamental e médio, mas diferiam em termos de dificuldade (por exemplo, com base no número de variáveis a serem consideradas em um padrão e no tamanho da matriz para problemas de abstração). Os itens de exemplo mostrados neste documento são da versão da avaliação para o ensino fundamental.

A avaliação para decomposição de problemas utiliza uma série de quebra-cabeças progressivamente mais difíceis, semelhantes ao jogo chamado Mastermind (o jogo de tabuleiro desenvolvido por Meiorowitz em 1970; ver <https://boardgamegeek.com/boardgame/2392/mastermind>) onde os alunos devem usar o feedback do jogo para descobrir quais valores por recurso gráfico (cor e/ou forma e/ou padrão) são necessários para resolver um quebra-cabeça (ver Fig. 3; o lado esquerdo é o problema inicial, e o lado direito é receber feedback sobre o teste atual). O aluno precisa arrastar um objeto de cada vez para a caixa de teste para encontrar a cor correta (e em níveis mais difíceis também a forma e o padrão) no menor número de tentativas possível. As escolhas incorretas são colocadas na caixa "não soluções". O feedback por objeto é representado por uma marca verde para cor/forma/padrão correto e um X vermelho para um atributo incorreto.

Para abstração, os quebra-cabeças de correspondência de padrões exigem que os alunos identifiquem uma regra subjacente. Os alunos arrastam objetos de um conjunto à direita para as células cinzas à esquerda para completar o padrão e, assim, aplicar a regra inferida. Cada forma colorida só pode aparecer uma vez na solução. Veja a Figura 4.

Para o design do algoritmo, os jogadores devem alinhar uma sequência de setas (com antecedência) para guiar um personagem ao longo de um caminho em um labirinto que atenda a critérios especificados. Na Figura 5, a tarefa de sequenciamento exige que o aluno insira setas direcionais, juntamente com o número de iterações necessárias, para guiar um duende até um pote de ouro, no menor número de passos possível, evitando certos obstáculos.

Finalmente, a avaliação CT para o estudo Zoombinis inclui um conjunto de itens gerais de reconhecimento de padrões (Matrizes Progressivas de Raven, RPM; Raven, 1981) como base da capacidade dos alunos de inferir e aplicar diferentes padrões em situações cada vez mais complexas. Raven (2000) apontou que o RPM concentra-se em dois componentes da capacidade cognitiva geral: capacidade reprodutiva e reprodutiva. A capacidade educativa envolve dar sentido ao caos e gerar esquemas de alto nível para lidar com a complexidade. A capacidade reprodutiva é a capacidade de recordar e reproduzir informações que foram explicadas. A aplicação de regras em geral é o comportamento de seguir as regras básicas de um problema para resolvê-lo. É uma expressão externa da percepção que os solucionadores têm das características dos problemas, através de habilidades educativas e reprodutivas. Um exemplo é mostrado na Fig.

Para cada item, os alunos clicam e arrastam a opção correta entre as oito opções possíveis na parte inferior para completar o padrão na parte superior, onde a solução se aplica horizontalmente, verticalmente e diagonalmente.

Estas avaliações esforçam-se por examinar os fundamentos cognitivos fundamentais da PC de uma forma abstrata e não contextual. A EdGE está usando essas avaliações em combinação com a mineração de dados educacionais de comportamentos de jogos em Zoombinis e observações da aprendizagem de TC em sala de aula para obter uma imagem completa da aprendizagem de TC dos alunos do 3º ao 8º ano.

5. Discussão

Nesta revisão, analisamos as definições existentes de PC, com foco na literatura que poderia ajudar a descrever a base conceitual do PC que deveria orientar a aprendizagem e a avaliação em ambientes de ensino fundamental e médio. Discutimos como a TC é semelhante e distinta de outras formas de pensar (por exemplo, pensamento matemático e pensamento sistêmico) e descrevemos as relações entre a TC

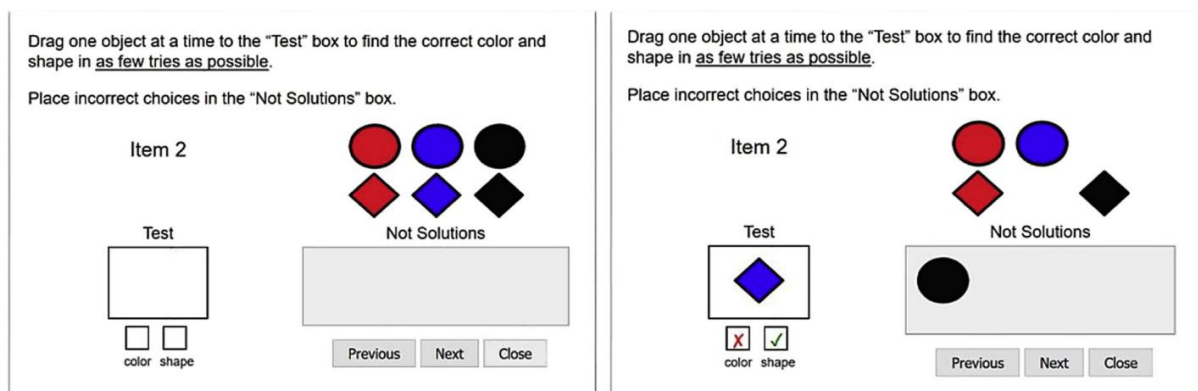


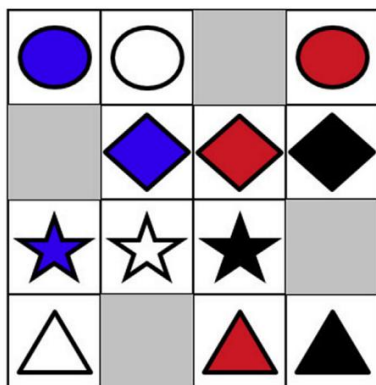
Figura 3. Item simples na avaliação do CT para decomposição do problema.

Item 3

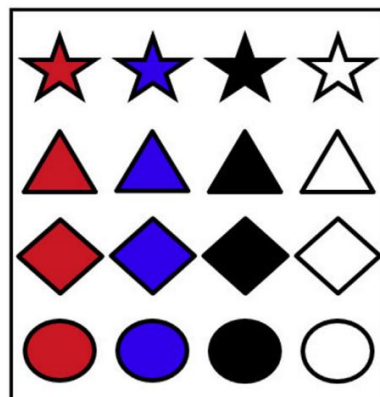
Drag objects from the box on the right into the gray cells to complete the pattern.

Each colored shape can only appear once in your solution.

Click "Submit" when you think you have it right.



Submit



Previous

Next

Close

Figura 4. Exemplo de item de avaliação de CT para abstração.

Item 1

Get the leprechaun to each colored box in order:
 1. **blue**
 2. **yellow**
 in as few moves as possible.
You cannot move over the same path twice!
Avoid the obstacles.

1 ▾ → 1 ▾ ← 1 ▾ ↑ 1 ▾ ↓

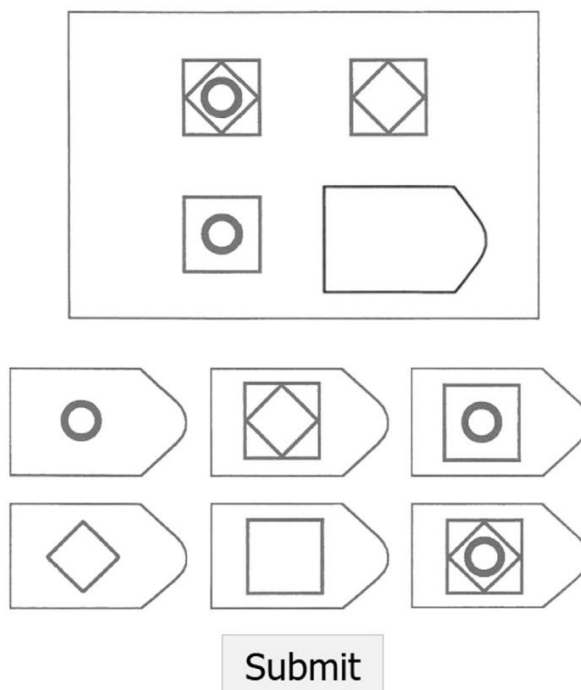
Run **Clear**

Previous Next Close

Figura 5. Item de sequenciamento simples/projeto algorítmico na avaliação de TC.

e ciência da computação. Revimos intervenções e avaliações utilizadas para desenvolver e medir as competências de PC dos alunos, desde o jardim de infância até ao ensino superior. Por fim, apresentamos exemplos de pesquisas que buscam medir o desenvolvimento do PC em experiências para alunos do ensino fundamental e médio. Nosso objetivo com este artigo de revisão foi sintetizar a literatura a fim de fornecer uma estrutura geral de TC, incluindo sua definição, facetas e elaboração por faceta, para desenvolver um modelo de TC que oriente a aprendizagem e avaliação de TC no ensino fundamental e médio.

Drag an option to the empty slot to complete the pattern and then hit the "Submit" button.



6. Exemplo de item de avaliação de TC da Raven para identificação de padrões.

Esta revisão da literatura esclareceu uma série de lacunas na pesquisa em TC. Por exemplo, falta na literatura uma definição geralmente aceita de TC, juntamente com uma especificação dos seus componentes (Wing, 2008). O uso inconsistente de termos ocorre em muitos artigos, como na combinação de ciência da computação, programação de computadores e CT (Czerkowski & Lyman, 2015; Israel et al., 2015).

A imaturidade da área que resulta nesta ambiguidade é agravada quando se olha para a educação. Os professores geralmente não estão familiarizados com a TC e têm dificuldade em encontrar ligações entre a TC e os seus currículos atuais. Conceber e desenvolver uma avaliação de TC fiável e válida é fundamental para o sucesso do ensino de TC integrado em múltiplas disciplinas (Grover & Pea, 2013). Mas a falta de uma definição padrão para operacionalizar a TC leva, consequentemente, a pesquisas onde as medidas variam muito entre os estudos, o que torna os resultados menos convincentes e certamente difíceis de comparar.

Outra questão que precisa ser resolvida diz respeito à dificuldade em julgar se as intervenções de TC em sala de aula realmente alcançam os resultados desejados (Settle et al., 2012). Mais uma vez, os investigadores apontaram a dificuldade de avaliar o PC nas salas de aula e apelaram à avaliação em tempo real para fornecer dados de progressão da aprendizagem por aluno para apoiar a instrução dos professores (Bers et al., 2014; Wolz, Stone, Pearson, Puliood e Suiça, 2011). O quadro que propusemos pode servir de orientação para desenvolver tarefas de avaliação que evidenciem competências específicas de PC. Os exemplos que apresentamos de como o CT está sendo medido no Scratch, Zoombinis e FACT pretendem destacar essas possibilidades.

Estudos sobre transferência de CT também são necessários (Bers et al., 2014). Um problema é como identificar a aplicação da TC em outros domínios (Czerkowski & Lyman, 2015). Esforços foram feitos por alguns pesquisadores, como Ioannidou et al. (2011), que tentaram examinar a transferência de PC adquiridos de jogos para cursos de matemática e ciências. Da mesma forma, Repenning et al. (2015) teve como objetivo investigar as habilidades de PC transferidas dos jogos para a criação de simulações de fenômenos científicos. Grover et al. (2015) descobriram que os alunos poderiam aplicar conceitos computacionais aprendidos no Scratch a linguagens de programação específicas baseadas em texto, o que é bastante promissor. Em qualquer caso, a retenção a longo prazo de competências de PC, juntamente com a aplicação de competências de PC a outros contextos e domínios, é pouco investigada.

Uma última área que merece atenção de investigação envolve as diferenças de género no desenvolvimento da PC. As mulheres são frequentemente sub-representadas em assuntos relacionados com STEM, especialmente quando chegam à faculdade. Os investigadores podem considerar a utilização da TC para motivar os alunos, especialmente as mulheres, para se especializarem em áreas científicas (por exemplo, Grover & Pea, 2013; Kazimoglu, Kiernan, Bacon, & Mackinnon, 2012; Repenning et al., 2015). No entanto, os resultados dos estudos que examinam as diferenças de género são inconsistentes. Atmatzidou e Demetriadis (2016) relataram que as competências de PC das raparigas melhoraram significativamente após a intervenção e que, em última análise, raparigas e rapazes atingiram o mesmo nível de competências. As meninas tendem a passar significativamente mais tempo aprendendo on-line depois da escola, resultando em

melhor desempenho do que os meninos (Grover et al., 2015). No entanto, nenhuma diferença de gênero foi relatada por Werner et al. (2012) e Yadav et al. (2014).

O pensamento computacional precisa ser desmistificado (por exemplo, Barr & Stephenson, 2011). Para tanto, desenvolvemos uma definição e mostramos exemplos de um modelo que: (a) considera a TC como uma forma lógica de pensar, e não simplesmente conhecer uma linguagem de programação; (b) concentra-se particularmente no desenvolvimento conceitual necessário para envolver-se na decomposição, abstração, design algorítmico, depuração, iteração e generalização de problemas; (c) examina as competências baseadas no desempenho relacionadas com cada uma destas facetas da TC, e (d) podem ser reforçadas e enfatizadas nos currículos existentes (por exemplo, STEM). Nosso modelo de TC apresentado neste artigo pretende ser amplamente aplicável, embora específico o suficiente para informar a medição do aprendizado de TC em geral e em nível de faceta (para fins de diagnóstico). Pesquisas atuais estão em andamento para validar esta afirmação. Prevemos que os próximos anos trarão muitos estudos empíricos para ajudar a refinar este modelo para uso em contextos abrangentes.

Reconhecimentos

Esta pesquisa não recebeu nenhum subsídio específico de agências de financiamento dos setores público, comercial ou sem fins lucrativos.

O coautor XXX agradece o financiamento da US National Science Foundation e o apoio intelectual da equipe EdGE at TERC. Este trabalho está mais intimamente relacionado com a concessão NSF/DRL/DRK12/#0150228.

Referências

- Allan, V., Barr, V., Brylow, D., & Hambrusch, S. (março de 2010). Pensamento computacional em cursos de ensino médio. Nos *Anais do 41º ACM técnico simpósio sobre educação em ciência da computação* (pp. 390e391). ACM.
- Anderson, ND (2016). Um apelo ao pensamento computacional na graduação em psicologia. *Aprendizagem e Ensino de Psicologia*, 15, 226e234. <http://dx.doi.org/10.1177/1475725716659252>.
- Atmatzidou, S. e Demetriadi, S. (2016). Aprimorando as habilidades de pensamento computacional dos alunos por meio da robótica educacional: um estudo sobre idade e gênero diferenças relevantes. *Robótica e Sistemas Autônomos*, 75, 661e670. <http://dx.doi.org/10.1016/j.robot.2015.10.008>.
- Baglioni, A. e Evangelou, D. (2016). Praticando engenharia enquanto constrói com blocos: identificando o pensamento de engenharia. *Educação Infantil Europeia* *Diário de Pesquisa*, 24(1), 67e85. <http://dx.doi.org/10.1080/1350293X.2015.1120521>.
- Barr, D., Harrison, J. e Conery, L. (2011). Pensamento computacional: uma habilidade da era digital para todos. *Aprendendo e liderando com tecnologia*, 38(6), 20e23.
- Barr, V. e Stephenson, C. (2011). Trazendo o pensamento computacional para o ensino fundamental e médio: o que está envolvido e qual o papel do ensino da ciência da computação comunidade? *Incurções de ACM*, 2, 48e54. <http://dx.doi.org/10.1145/1929887.1929905>.
- Basawapatna, A., Koh, KH, Repenning, A., Webb, DC, & Marshall, KS (2011, março). Reconhecendo padrões de pensamento computacional. Em *Anais do 42º Simpósio Técnico ACM sobre Ensino de Ciência da Computação* (pp. 245e250). ACM.
- Basu, S., Biswas, G. e Kinnebrew, JS (2017). Modelagem de alunos para andaimes adaptativos em um ambiente de aprendizagem científica baseado em pensamento computacional. *Modelagem de Usuário e Interação Adaptada ao Usuário*, 27(1), 5e53.
- Berland, M. e Wilensky, U. (2015). Comparando ambientes robóticos virtuais e físicos para suportar sistemas complexos e pensamento computacional. *Revista de Educação Científica e Tecnologia*, 24, 628e647. <http://dx.doi.org/10.1007/s10956-015-9552-x>.
- Bers, M., Flannery, L., Kazakoff, E., & Sullivan, A. (2014). Pensamento e ajustes computacionais: Exploração de um currículo de robótica para a primeira infância. *Computadores e Educação*, 72, 145e157. <http://dx.doi.org/10.1016/j.compedu.2013.10.02>.
- Brennan, K. e Resnick, M. (abril de 2012). Novas estruturas para estudar e avaliar o desenvolvimento do pensamento computacional. Nos *Anais de 2012 reunião anual da associação americana de pesquisa educacional* (Vancouver, Canadá).
- Burke, Q., O'Byrne, WI e Kafai, YB (2016). Participação computacional: Compreendendo a codificação como uma extensão da instrução de alfabetização. *Diário de Alfabetização de Adolescentes e Adultos*, 59, 371e375. <http://dx.doi.org/10.1002/jaal.496>.
- Cetin, I. (2016). Introdução à computação para professores de formação inicial: explorando a utilização do zero. *Jornal de Pesquisa em Computação Educacional*. <http://dx.doi.org/10.1177/0735633116642774>. Publicação online avançada.
- Cuny, J., Snyder, L. e Wing, JM (2010). Desmistificando o pensamento computacional para não cientistas da computação. Manuscrito não publicado, referenciado em <http://www.cs.cmu.edu/~CompThink/resources/TheLinkWing.pdf>.
- Czerkawski, BC e Lyman, EW, III (2015). Explorando questões sobre pensamento computacional no ensino superior. *Tendências tecnológicas*, 59(2), 57e65. <http://dx.doi.org/10.1007/s11528-015-0840-3>.
- Denner, J., Werner, L., Campe, S., & Ortiz, E. (2014). Programação em pares: em que condições é vantajosa para alunos do ensino médio? *Jornal de Pesquisa em Tecnologia na Educação*, 46, 277e296. <http://dx.doi.org/10.1080/15391523.2014.888272>.
- Grover, S. (abril de 2011). Robótica e engenharia para alunos do ensino fundamental e médio desenvolverem o pensamento computacional. In *Artigo apresentado na reunião anual da American Educational Research Association*. Nova Orleans, Los Angeles.
- Grover, S. e Pea, R. (2013). Pensamento computacional em Ke12: Uma revisão do estado do campo. *Pesquisador Educacional*, 42(1), 38e43. <http://dx.doi.org/10.3102/0013189X12463051>.
- Grover, S., Pea, R. e Cooper, S. (2015). Projetando para um aprendizado mais profundo em um curso misto de ciência da computação para alunos do ensino médio. *Educação em Ciência da Computação*, 25, 199e237. <http://dx.doi.org/10.1080/08993408.2015.1033142>.
- Harel, G. e Sowder, L. (2005). Pensamento matemático avançado em qualquer idade: sua natureza e seu desenvolvimento. *Pensamento e Aprendizagem Matemática*, 7(1), 27e50. http://dx.doi.org/10.1207/s15327833m0701_3.
- Ioannidou, A., Bennett, V., Repenning, A., Koh, KH e Basawapatna, A. (abril de 2011). Padrões de pensamento computacional. In *Artigo apresentado na reunião anual da American Educational Research Association*. Nova Orleans, Los Angeles.
- Israel, M., Pearson, J., Tapia, T., Werfel, Q., & Reese, G. (2015). Apoiando todos os alunos no pensamento computacional em toda a escola: uma análise qualitativa entre casos. *Computadores e Educação*, 82, 263e279. <http://dx.doi.org/10.1016/j.compedu.2014.11.022>.
- Jun, S., Han, S., Kim, H. e Lee, W. (2014). Avaliando a alfabetização computacional de alunos do ensino fundamental em nível nacional na Coreia. *Avaliação Educacional, Avaliação e Responsabilidade*, 26, 319e332. <http://dx.doi.org/10.1007/s11092-013-9185-7>.
- Kazimoglu, C., Kiernan, M., Bacon, L. e Mackinnon, L. (2012). Um jogo sério para desenvolver o pensamento computacional e aprender programação introdutória de computadores. *Procedia-Ciências Sociais e Comportamentais*, 47, 1991e1999.
- Kim, B., Kim, T. e Kim, J. (2013). Estratégia de programação de papel e lápis em direção ao pensamento computacional para não-graduados: projete sua solução. *Diário de Pesquisa em Computação Educacional*, 49, 437e459. <http://dx.doi.org/10.2190/EC.49.4.b>.
- Kim, YC, Kwon, DY e Lee, WG (2014). Modelagem e simulação computacional para aprendizado de um conceito de automação em curso de programação. *Revista Internacional de Teoria e Engenharia da Computação*, 6, 341e345. <http://dx.doi.org/10.7763/IJCTE.2014.V6.886>.
- Korkmaz, O., Çakır, R., & Ozden, MY (2017). Um estudo de validade e confiabilidade das escalas de pensamento computacional (CTS). *Computadores no Comportamento Humano*, 72, 558e569.
- Lu, JJ e Fletcher, GH (março de 2009). Pensando em pensamento computacional. *Boletim ACM SIGCSE*, 41(1), 260e264.

- Lye, S. e Koh, J. (2014). Revisão sobre ensino e aprendizagem de pensamento computacional por meio de programação: o que vem por aí para o ensino fundamental e médio? *Computadores em humanos Comportamento*, 41, 51e61. <http://dx.doi.org/10.1016/j.chb.2014.09.012>.
- Mishra, P. e Yadav, A. (2013). Da arte e dos algoritmos: repensando a tecnologia e a criatividade no século XXI. *Tendências tecnológicas*, 57(3), 10e14.
- Conselho Nacional de Pesquisa. (2010). Comitê para Workshops de Pensamento Computacional: Relatório de um workshop sobre o escopo e a natureza do pensamento computacional pensamento. Washington, DC: Imprensa das Academias Nacionais.
- Papert, S. (1980). *Tempestades mentais. Crianças, computadores e ideias poderosas*. Nova York: Livros básicos.
- Papert, S. (1991). Situando o construcionismo. Em S. Papert, & I. Harel (Eds.), *Construcionismo*. Cambridge, MA: MIT Press.
- Pawley, A. (2009). Narrativas universalizadas: padrões de como os docentes definem "engenharia". *Revista de Educação em Engenharia*, 98, 309e319.
- Raven, JC (1981). *Manual para matrizes progressivas e escalas de vocabulário de Raven*. Suplemento de pesquisa nº 1: A padronização britânica de 1979 das matrizes progressivas padrão e das escalas de vocabulário de Mill Hill, juntamente com dados comparativos de estudos anteriores no Reino Unido, EUA, Canadá, Alemanha e Irlanda. San Antonio, TX: Avaliação Harcourt.
- Raven, JC (2000). As matrizes progressivas do Raven: Mudança e estabilidade ao longo da cultura e do tempo. *Psicologia Cognitiva*, 41, 1e48.
- Razzouk, R. e Shute, V. (2012). O que é design thinking e por que é importante? *Revisão de Pesquisa Educacional*, 82, 330e348.
- Repenning, A., Webb, DC, Koh, KH, Nickerson, H., Miller, SB, Brand, C.,...Repenning, N. (2015). Design de jogos escalável: uma estratégia para levar o ensino sistêmico de ciência da computação às escolas por meio do design de jogos e da criação de simulações. *Transações ACM em Educação em Computação (TOCE)*, 15(2), 1e31. <http://dx.doi.org/10.1145/2700517>.
- Rom an-Gonz alez, M., Perez-Gonz alez, JC, & Jimenez-Fern andez, C. (2017). Quais habilidades cognitivas estão subjacentes ao pensamento computacional? Validade de critério de o teste de pensamento computacional. *Computadores no Comportamento Humano*, 72, 678e691.
- Rowe, E., Asbell-Clarke, J., & Baker, RS (2015). Análise de jogos sérios para medir a aprendizagem científica implícita. Em CS Loh, Y. Sheng e D. Ifenthaler (Eds.), *Análise de jogos sérios: metodologias para medição, avaliação e melhoria de desempenho* (pp. 343e360). Suíça: Springer International Publishing.
- Sanford, JF e Naidu, JT (2016). Conceitos de pensamento computacional para o ensino fundamental. *Questões Contemporâneas em Pesquisa em Educação (CIER)*, 9(1), 23e31. <http://dx.doi.org/10.19030/cier.v9i1.9547>.
- Seiter, L., & Foreman, B. (2013, agosto). Modelagem das progressões de aprendizagem do pensamento computacional de alunos do ensino fundamental. Em *Anais do nono conferência internacional anual da ACM sobre pesquisa internacional em educação em computação* (pp. 59e66). ACM.
- Sengupta, P., Kinnebrew, JS, Basu, S., Biswas, G. e Clark, D. (2013). Integrando o pensamento computacional com o ensino de ciências K-12 usando computação baseada em agentes: uma estrutura teórica. *Educação e Tecnologias de Informação*, 18, 351e380. <http://dx.doi.org/10.1007/s10639-012-9240-x>.
- Settle, A., Franke, B., Hansen, R., Spaltro, F., Jurisson, C., Rennert-May, C., et al. (2012, julho). Infundindo pensamento computacional no currículo do ensino fundamental e médio . Nos *Anais da 17ª Conferência Anual da ACM sobre Inovação e Tecnologia no Ensino de Ciência da Computação* (pp. 22e27). ACM.
- Shute, VJ (1991). Quem provavelmente adquirirá habilidades de programação? *Jornal de Pesquisa em Computação Educacional*, 7(1), 1e24.
- Shute, VJ, Masduki, I. e Donmez, O. (2010). Estrutura conceitual para modelar, avaliar e apoiar competências em ambientes de jogos. *Tecnologia, Instrução, Cognição e Aprendizagem*, 8, 137e161.
- Sneider, C., Stephenson, C., Schafer, B. e Flick, L. (2014). Pensamento computacional em salas de aula de ciências do ensino médio. *O Professor de Ciências*, 81(5), 53e59.
- Weintrop, D., Beheshti, E., Horn, M., Orton, K., Jona, K., Trouille, L., et al. (2016). Definindo pensamento computacional para salas de aula de matemática e ciências. *Revista de Educação Científica e Tecnologia*, 25(1), 127e147. <http://dx.doi.org/10.1007/s10956-015-9581-5>.
- Werner, L., Denner, J., Campe, S. e Kawamoto, DC (fevereiro de 2012). A avaliação de desempenho das fadas: Medindo o pensamento computacional no ensino médio. Em *Anais do 43º Simpósio Técnico ACM sobre Educação em Ciência da Computação* (pp. 215e220). ACM.
- Asa, JM (2006). Pensamento computacional. *Comunicações da ACM*, 49(3), 33e35.
- Asa, JM (2008). Pensamento computacional e pensamento sobre computação. *Transações Filosóficas da Royal Society a: Matemática, Física e Ciências de Engenharia*, 366, 3717e3725. <http://dx.doi.org/10.1098/rsta.2008.0118>.
- Asa, JM (2010). Pensamento computacional: o quê e por quê?. Manuscrito não publicado. Pittsburgh, PA: Departamento de Ciência da Computação, Carnegie Mellon Universidade. Obtido em <https://www.cs.cmu.edu/~CompThink/resources/TheLinkWing.pdf>.
- Wolz, U., Stone, M., Pearson, K., Puliod, SM e Switzer, M. (2011). Pensamento computacional e escrita expositiva no ensino médio. *Transações ACM sobre Educação em Computação (TOCE)*, 11(2), 1e22.
- Yadav, A., Mayfield, C., Zhou, N., Hambrusch, S., & Korb, JT (2014). Pensamento computacional na formação de professores do ensino fundamental e médio. *ACM Transações em Educação em Computação (TOCE)*, 14(1), 1e16. <http://dx.doi.org/10.1145/2576872>.
- Zhong, B., Wang, Q., Chen, J. e Li, Y. (2016). Uma exploração da avaliação integrada tridimensional para o pensamento computacional. *Revista Educacional Pesquisa de Computação*, 53, 562e590. <http://dx.doi.org/10.1177/0735633115608444>.